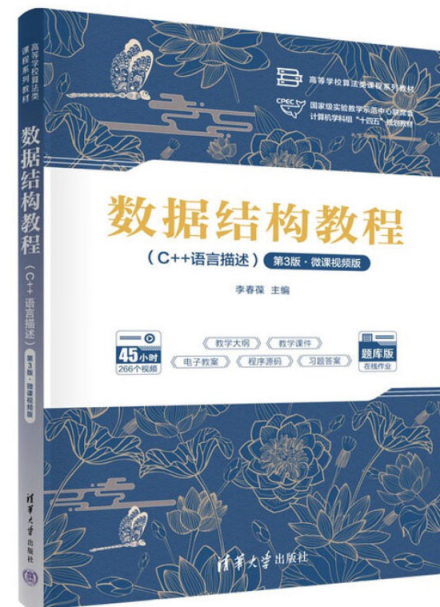
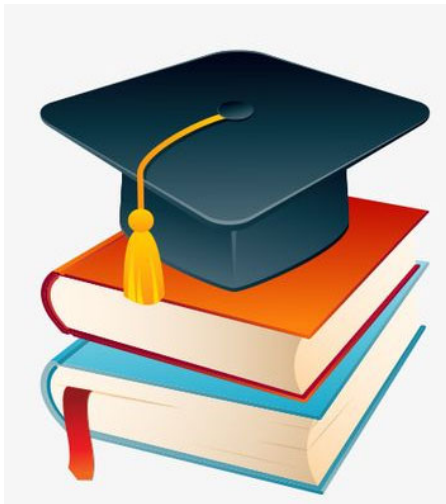


教材:

ISBN: 9787302685340



第1章 绪论

提纲

CONTENTS

1.1 什么是数据结构

1.2 算法及其描述

1.3 算法分析

1.4 数据结构的目标

1.1 什么是数据结构

1.1.1 数据结构的定义

用计算机解决一个具体问题的步骤

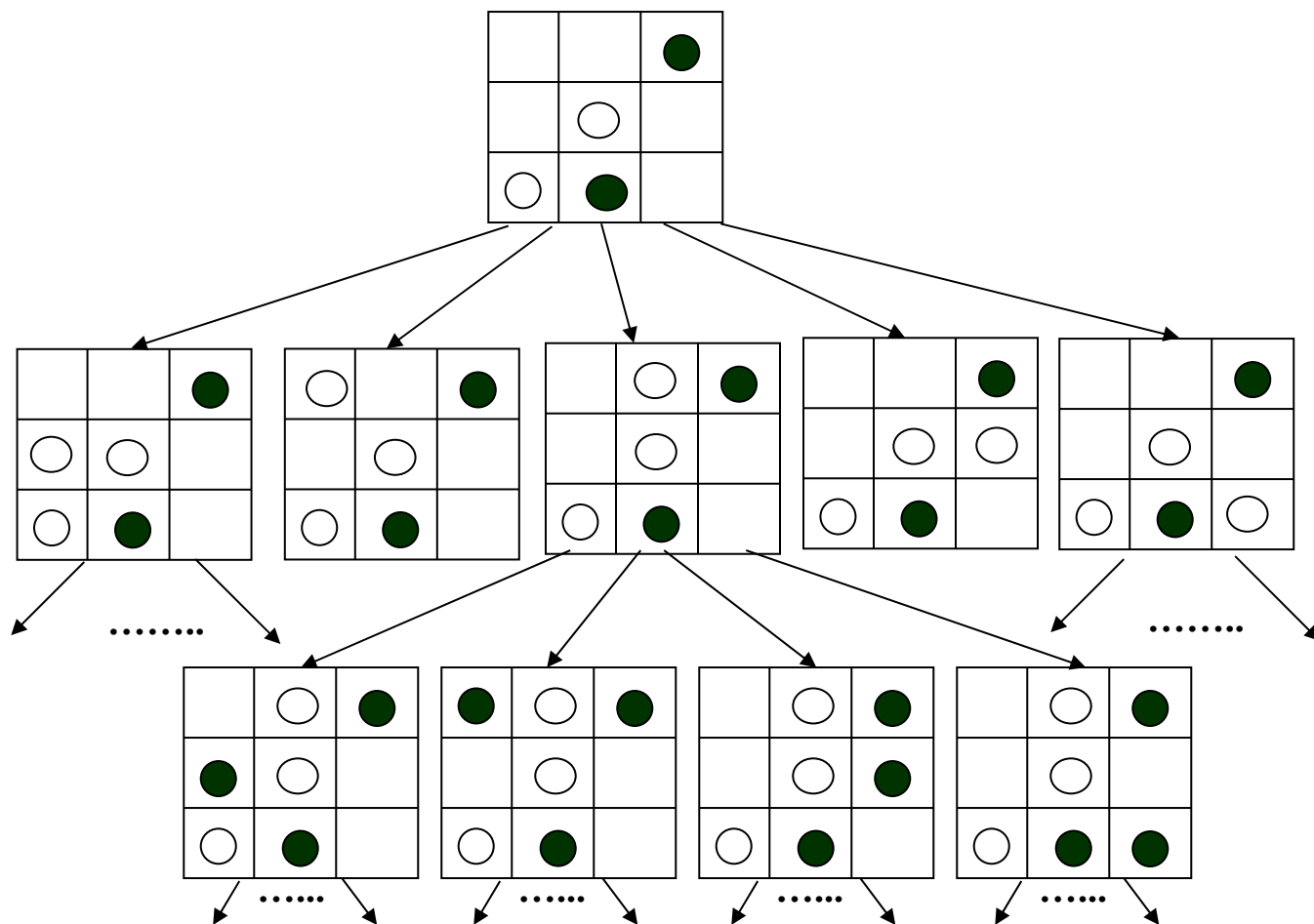
- (1) 分析问题，确定数据模型。
- (2) 设计相应的算法。
- (3) 编写程序，运行并调试程序直至得到正确的结果。

图书馆的书目检索自动化问题

线性表

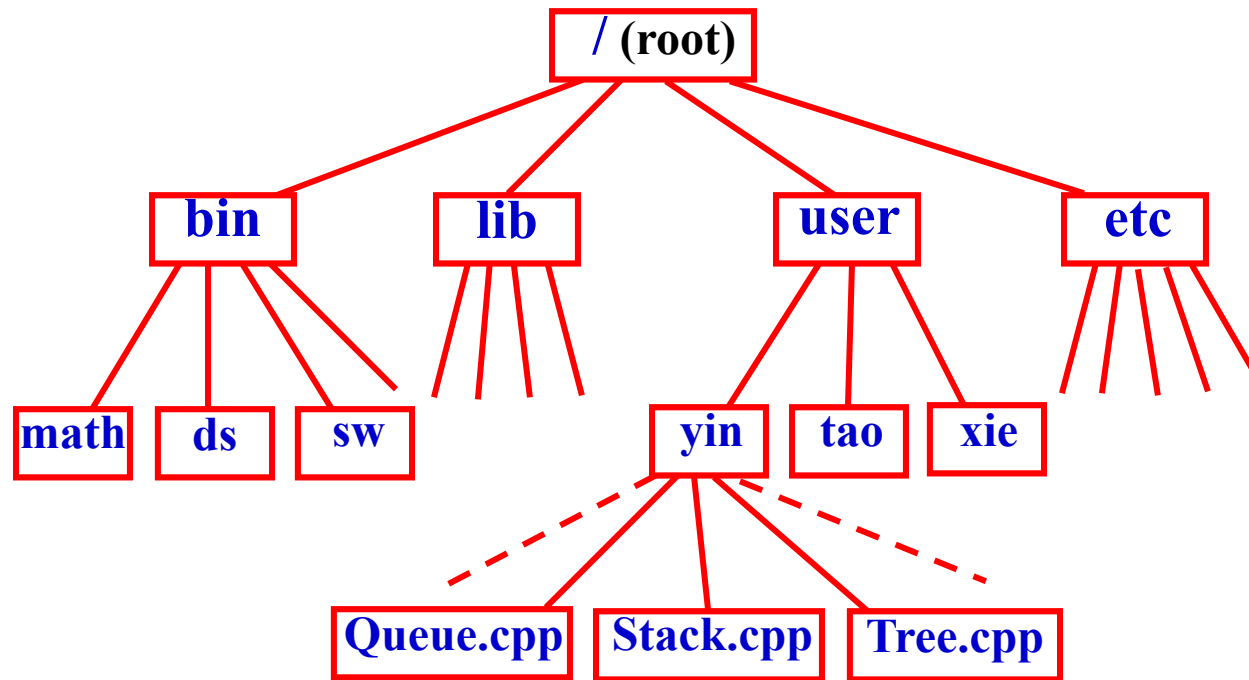


人机对奕问题

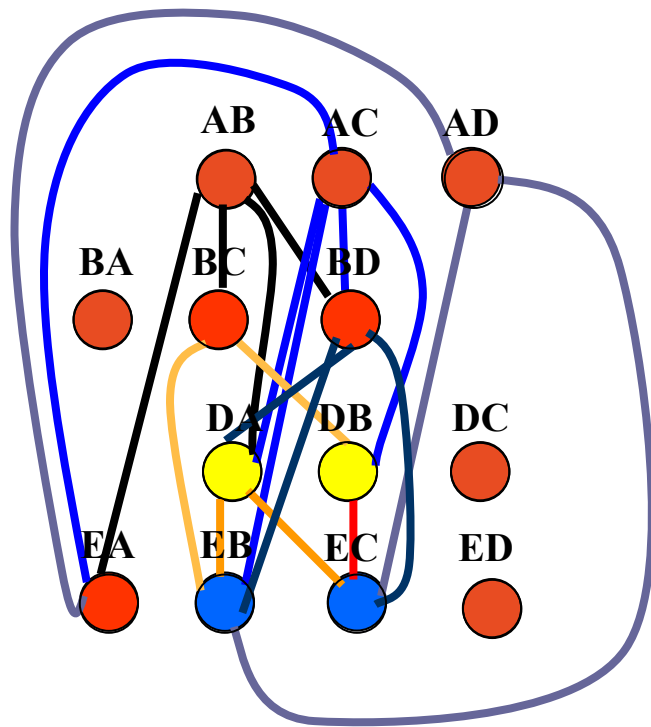


文件系统的系统结构图

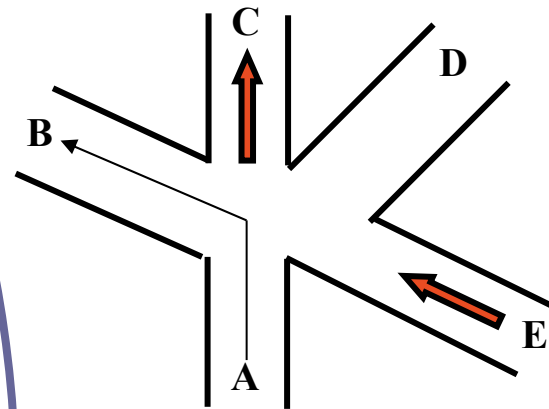
树



多叉路口交通灯管理问题



表示通路和图



五岔路口

顶点：一条通路

连线：不能同时通行

染色：有连线的两个顶点不能具有相同颜色

📖 求解非数值计算的问题：

设计出合适的数据结构及相应的算法

即：首先要考虑对相关的各种信息如何表示、组织和存储？

需要从数据入手来分析并得到解决问题的方法

数据是描述客观事物的数、字符以及所有能输入到计算机中并被计算机程序处理的符号的集合。

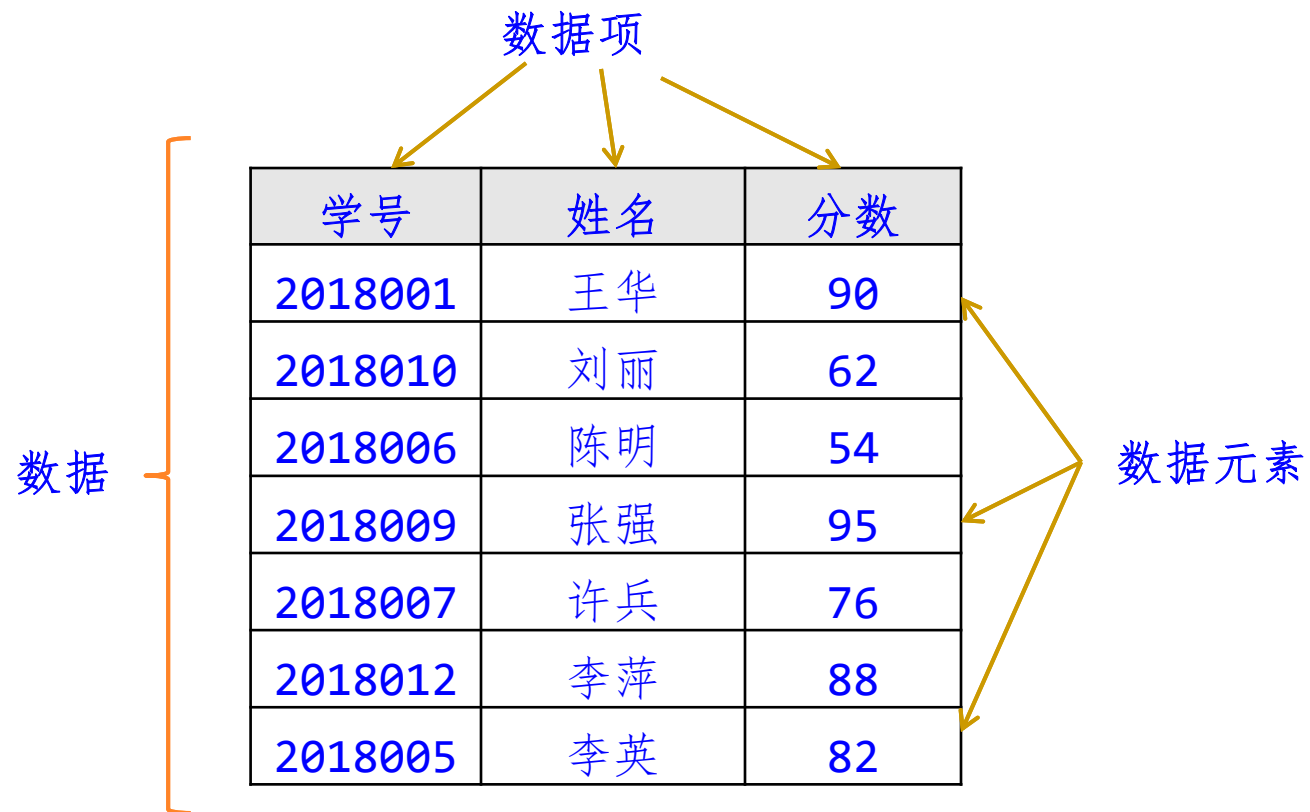
数据元素是数据的基本单位（例如，A班中的每个学生记录都是一个数据元素），也就是说数据元素是组成数据的、有一定意义的基本单位，在计算机中通常作为整体处理。

数据项是具有独立含义的数据最小单位，也称为成员或域（例如，A班中每个数据元素即学生记录是由学号、姓名、性别和班号等数据项组成）。



结构化数据

高等数学成绩表



数据对象是性质相同的有限个数据元素的集合，它是数据的一个子集。

如大写字母数据对象是集合 $C=\{'A', 'B', 'C', \dots, 'Z'\}$;
1~100的整数数据对象是集合 $N=\{1, 2, \dots, 100\}$ 。

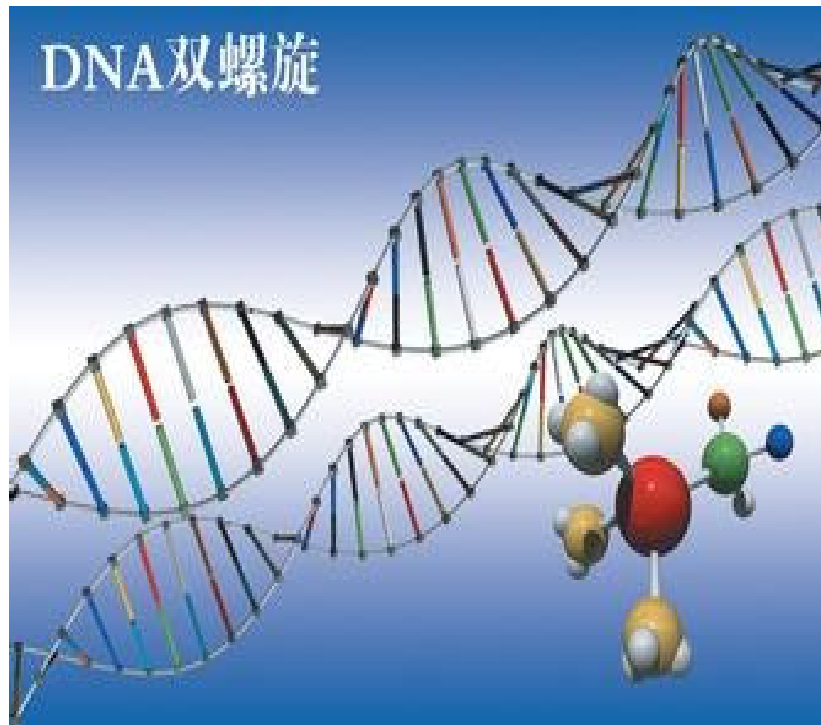
默认情况下，数据结构中的数据都指的是数据对象。

数据结构是指所涉及的数据元素以及数据元素之间的关系，可以看作是相互之间存在着特定关系的数据元素的集合。可以把数据结构看成是带结构的数据元素的集合。



数据元素之间的关系 $\Rightarrow$ 结构,现实世界的结构是纷繁复杂的

① 微观世界—DNA结构



② 宏观世界一建筑物的结构



数据结构中讨论的元素关系主要是指**相邻关系**或**邻接关系**。

学号	姓名	分数	
2018001	王华	90	相邻
2018010	刘丽	62	
2018006	陈明	54	
2018009	张强	95	
2018007	许兵	76	不相邻
2018012	李萍	88	
2018005	李英	82	

一个数据结构的几个方面：



- 数据元素之间的逻辑关系 $\Rightarrow$ 数据的逻辑结构。
- 数据元素及其关系在计算机存储器中的存储方式 $\Rightarrow$ 数据的存储结构（或物理结构）。
- 施加在该数据上的操作 $\Rightarrow$ 数据运算。

1.1.2 数据的逻辑结构

- 数据的逻辑结构是面向用户的，它反映数据元素之间的逻辑关系而不是物理关系。
- 数据的逻辑结构是独立于计算机的。

1. 逻辑结构的表示

由于数据逻辑结构是面向用户的，可以采用表格、图等用户容易理解的形式表示。

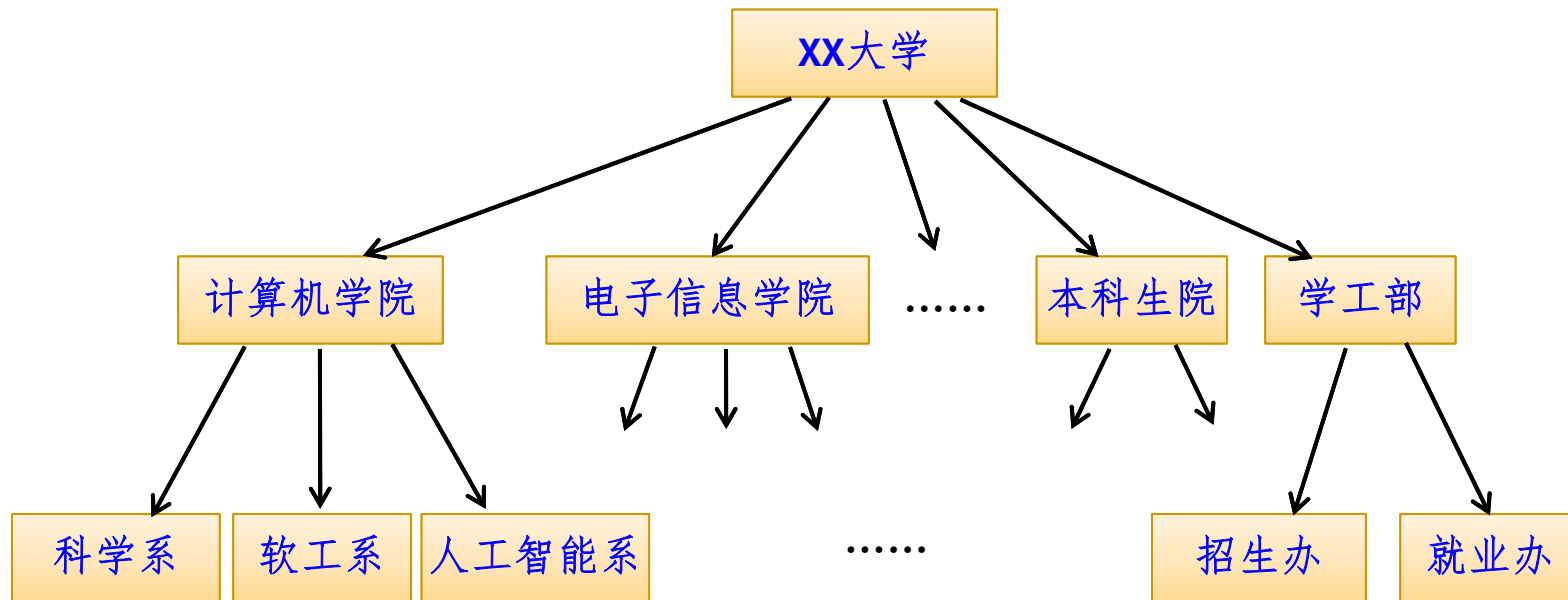
高等数学成绩表

例1.1

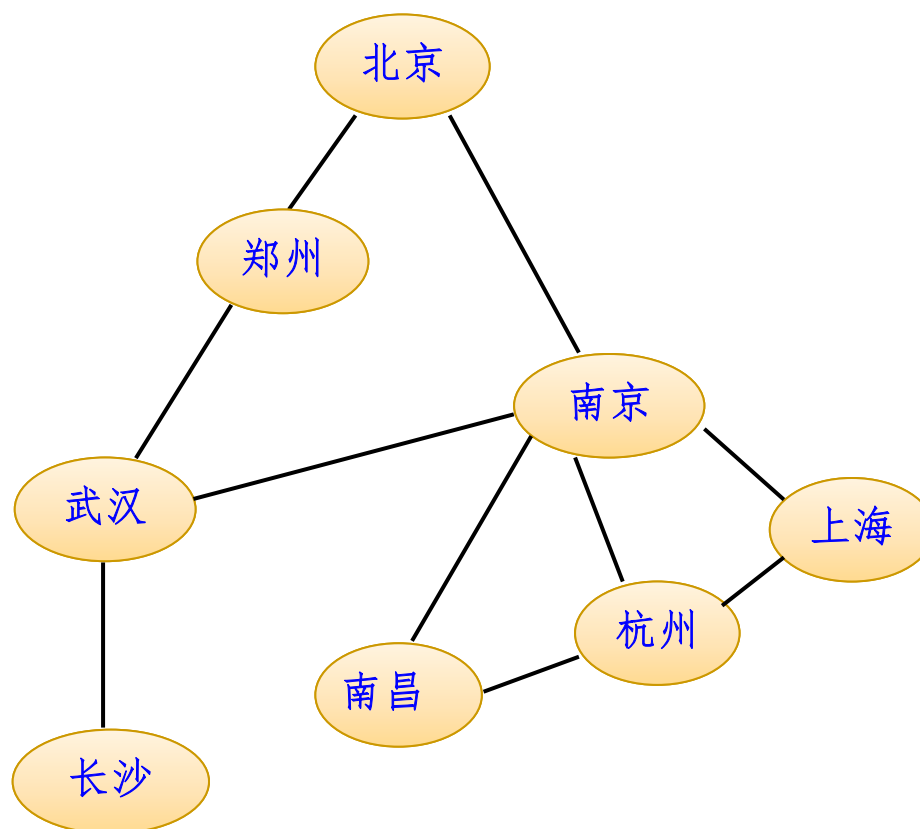
学号	姓名	分数
2018001	王华	90
2018010	刘丽	62
2018006	陈明	54
2018009	张强	95
2018007	许兵	76
2018012	李萍	88
2018005	李英	82



例1.2



例1.3



为了更通用地描述数据的逻辑结构，通常采用二元组表示数据的逻辑结构，一个**二元组**如下：

$$B=(D, R)$$

其中，**B**是一种逻辑数据结构，**D**是数据元素的集合，在**D**上数据元素之间可能存在多种关系，**R**是所有关系的集合。即：

$$D=\{d_i \mid 0 \leq i \leq n-1, n \geq 0\}$$

$$R=\{r_j \mid 1 \leq j \leq m, m \geq 0\}$$

$$R = \{r_j \mid 1 \leq j \leq m, m \geq 0\}$$

- R 中的某个关系 r_j ($1 \leq j \leq m$) 是序偶的集合。
- 对于 r_j 中的任一序偶 $\langle x, y \rangle$ ($x, y \in D$)，把 x 叫做序偶的第一元素，把 y 叫做序偶的第二元素，又称序偶的第一元素为第二元素的前驱元素，称第二元素为第一元素的后继元素。如在 $\langle x, y \rangle$ 的序偶中， x 为 y 的前驱元素，而 y 为 x 的后继元素。
- 若某个元素没有前驱元素，则称该元素为开始元素；若某个元素没有后继元素，则称该元素为终端元素。
- 对于对称序偶，即满足这样的条件：若 $\langle x, y \rangle \in r$ ($r \in R$)，则 $\langle y, x \rangle \in r$ ($x, y \in D$)，可用圆括号代替尖括号，即 $(x, y) \in r$ 。

- R 中的某个关系 r_j ($1 \leq j \leq m$) 是序偶的集合。
- 对于 r_j 中的任一序偶 $\langle x, y \rangle$ ($x, y \in D$)，把 x 叫做序偶的第一元素，把 y 叫做序偶的第二元素，又称序偶的第一元素为第二元素的前驱元素，称第二元素为第一元素的后继元素。如在 $\langle x, y \rangle$ 的序偶中， x 为 y 的前驱元素，而 y 为 x 的后继元素。
- 若某个元素没有前驱元素，则称该元素为开始元素；若某个元素没有后继元素，则称该元素为终端元素。
- 对于对称序偶，即满足这样的条件：若 $\langle x, y \rangle \in r$ ($r \in R$)，则 $\langle y, x \rangle \in r$ ($x, y \in D$)，可用圆括号代替尖括号，即 $(x, y) \in r$ 。

【例1.4】 采用二元组表示前面3个例子的逻辑结构。

解：高等数学成绩表(假设学号为关键字)的二元组表示如下：

$$B_1 = (D, R)$$

$$D = \{2018001, 2018010, 2018006, 2018009, 2018007, 2018012, 2018005\}$$

$$R = \{r_1\} \quad // \text{表示只有一种逻辑关系}$$

$$R = \{ \langle 2018001, 2018010 \rangle, \langle 2018010, 2018006 \rangle, \langle 2018006, 2018009 \rangle, \\ \langle 2018009, 2018007 \rangle, \langle 2018007, 2018012 \rangle, \langle 2018012, 2018005 \rangle \}$$

【例1.4】 采用二元组表示前面3个例子的逻辑结构。

解：某大学组织结构(假设单位名为关键字)的二元组表示如下：

$$B_2 = (D, R)$$

$D = (\text{XX大学}, \text{计算机学院}, \text{电子信息学院}, \dots, \text{本科生院}, \text{学工部}, \dots, \text{科学系}, \text{软工系}, \text{人工智能系}, \dots, \text{招生办}, \text{就业办})$

$$R = \{r_1\}$$

$r_1 = \{ \langle \text{XX大学}, \text{计算机学院} \rangle, \langle \text{XX大学}, \text{电子信息学院} \rangle, \dots, \langle \text{XX大学}, \text{本科生院} \rangle, \langle \text{XX大学}, \text{学工部} \rangle, \dots, \langle \text{计算机学院}, \text{科学系} \rangle, \langle \text{计算机学院}, \text{软工系} \rangle, \langle \text{计算机学院}, \text{人工智能系} \rangle, \dots, \langle \text{学工部}, \text{招生办} \rangle, \langle \text{学工部}, \text{就业办} \rangle \}$

【例1.4】 采用二元组表示前面3个例子的逻辑结构。

解：全国部分城市交通图(假设城市名为关键字)的二元组表示如下：

$$B_3 = (D, R)$$

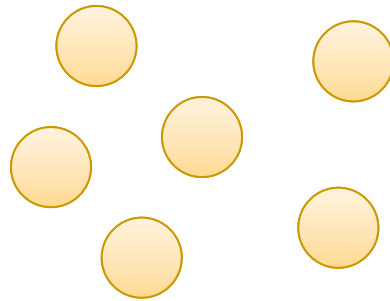
$$D = \{\text{北京, 郑州, 武汉, 长沙, 南京, 南昌, 杭州, 上海}\}$$

$$R = \{r_1\}$$

$$r_1 = \{(\text{北京, 郑州}), (\text{北京, 南京}), (\text{郑州, 武汉}), (\text{武汉, 南京}), (\text{武汉, 长沙}), (\text{南京, 南昌}), (\text{南京, 上海}), (\text{南京, 杭州}), (\text{南昌, 杭州}), (\text{杭州, 上海})\}$$

2. 逻辑结构的类型

- **集合：**结构中数据元素之间除了“同属于一个集合”的关系外，没有其他关系，与数学中的集合概念相同。



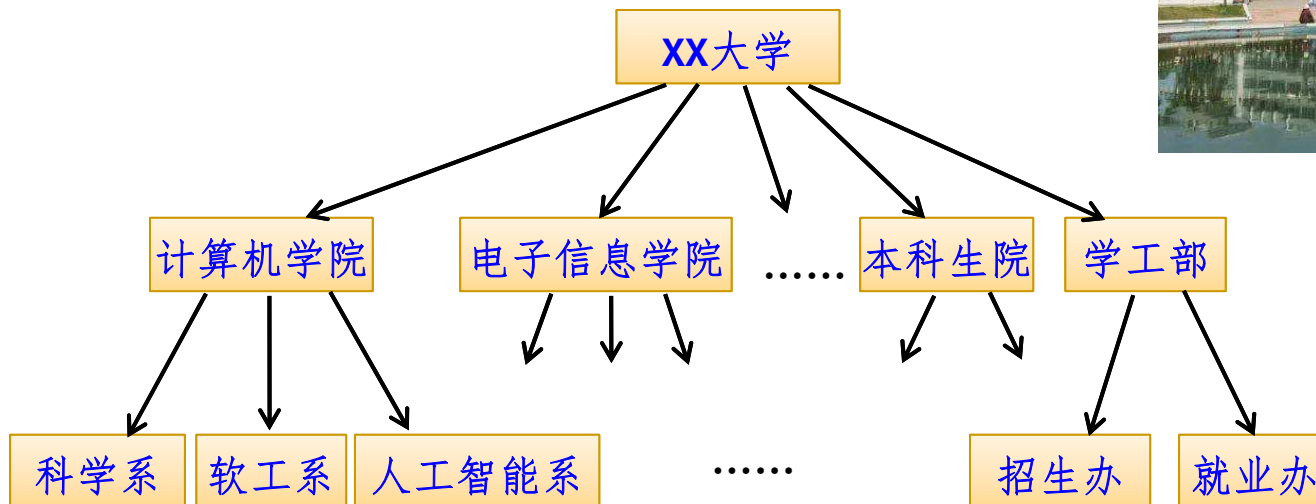
■ **线性结构：**若结构是非空的，则有且仅有一个开始元素和终端元素，并且所有元素最多只有一个前驱元素和一个后继元素。

高等数学成绩表

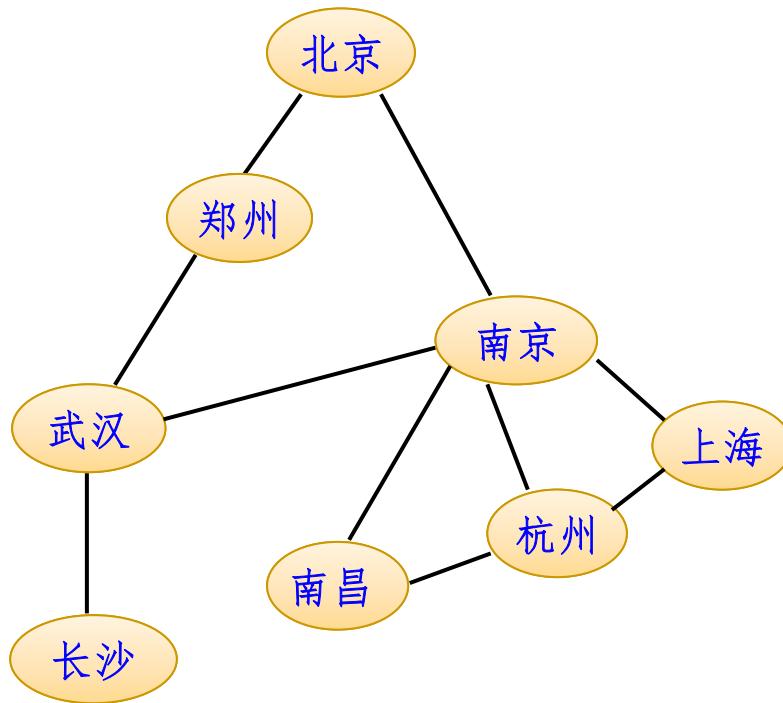
学号	姓名	分数
2018001	王华	90
2018010	刘丽	62
2018006	陈明	54
2018009	张强	95
2018007	许兵	76
2018012	李萍	88
2018005	李英	82



- **树形结构：**若结构是非空的，则有且仅有一个元素为开始元素（也称为根结点），可以有多个终端元素，每个元素有零个或多个后继元素，除开始元素外每个元素有且仅有一个前驱元素。

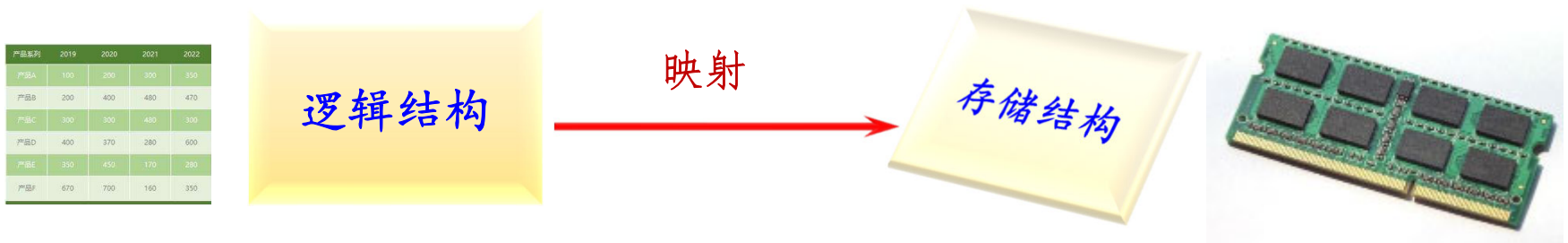


■ **图形结构：**若结构是非空的，则每个元素可以有多个前驱元素和多个后继元素。



1.1.3 数据的存储结构

数据在计算机存储器中的存储方式就是存储结构。它是面向程序员的。



设计存储结构的这种映射应满足两个要求：

- 存储所有元素
- 存储数据元素间的关系

【例1.5】 对于表1.1所示高等数学成绩表，设计多种存储结构，并讨论各种存储结构的特性。

高等数学成绩表

学号	姓名	分数
2018001	王华	90
2018010	刘丽	62
2018006	陈明	54
2018009	张强	95
2018007	许兵	76
2018012	李萍	88
2018005	李英	82

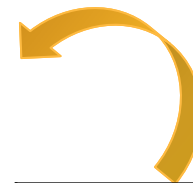
存储结构1: 用C++语言中的结构体数组来存储高等数学成绩表，设计其元素类型Stud1如下：

```
struct Stud1                                //学生成绩元素类型
{
    int no;                                //存放学号
    string name;                            //存放姓名
    int score;                             //存放分数

    Stud1() {}                             //构造函数
    Stud1(int no1,string name1,int score1) //重载构造函数
    {
        no=no1;
        name=name1;
        score=score1;
    }
};
```

用**data**数组（所有的元素类型均为**Stud1**）存放高等数学成绩表：

```
void Create()           //创建高数成绩顺序表
{
    data[0]=Stud1(2018001,"王华",90);
    data[1]=Stud1(2018010,"刘丽",62);
    data[2]=Stud1(2018006,"陈明",54);
    data[3]=Stud1(2018009,"张强",95);
    data[4]=Stud1(2018007,"许兵",76);
    data[5]=Stud1(2018012,"李萍",88);
    data[6]=Stud1(2018005,"李英",82);
    length=7;
}
```



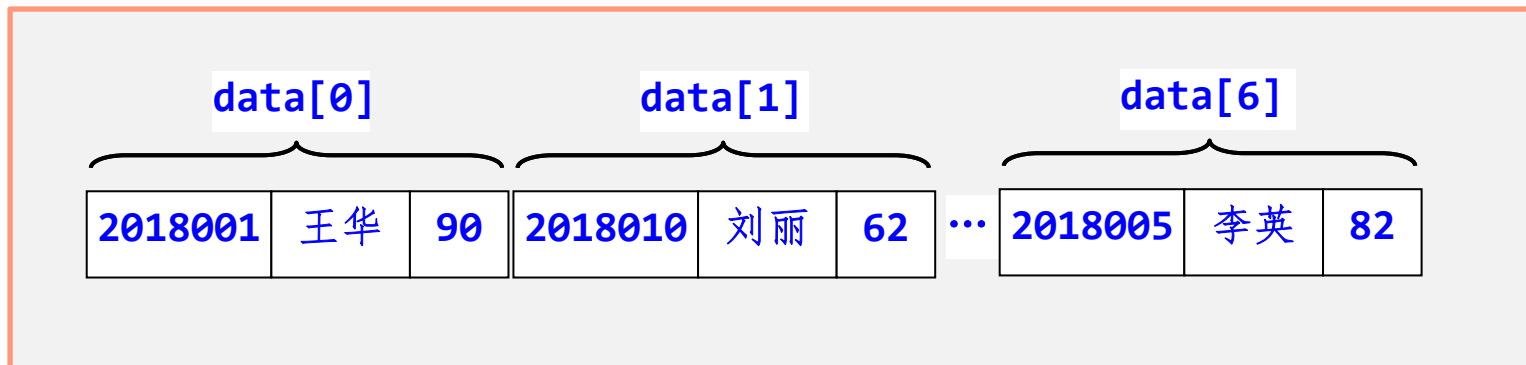
学号	姓名	分数
2018001	王华	90
2018010	刘丽	62
2018006	陈明	54
2018009	张强	95
2018007	许兵	76
2018012	李萍	88
2018005	李英	82

学号	姓名	分数
2018001	王华	90
2018010	刘丽	62
2018006	陈明	54
2018009	张强	95
2018007	许兵	76
2018012	李萍	88
2018005	李英	82



映射





该存储结构的特性：

- 所有元素存放在一片地址连续的存储单元中。
- 逻辑上相邻的元素在物理位置上也是相邻的，所以不需要额外空间表示元素之间的逻辑关系。

称为**顺序存储结构**。

存储结构2: 用C++语言中的单链表来存储高等数学成绩表，设计其结点类型Stud2如下：

```
struct Stud2                                //学生单链表结点类型
{
    int no;                                //存放学号
    string name;                            //存放姓名
    int score;                             //存放分数
    Stud2* next;                           //存放下一个结点指针

    Stud2(int no1,string name1,int score1) //重载构造函数
    {
        no=no;
        name=name1;
        score=score1;
        next=NULL;
    }
};
```

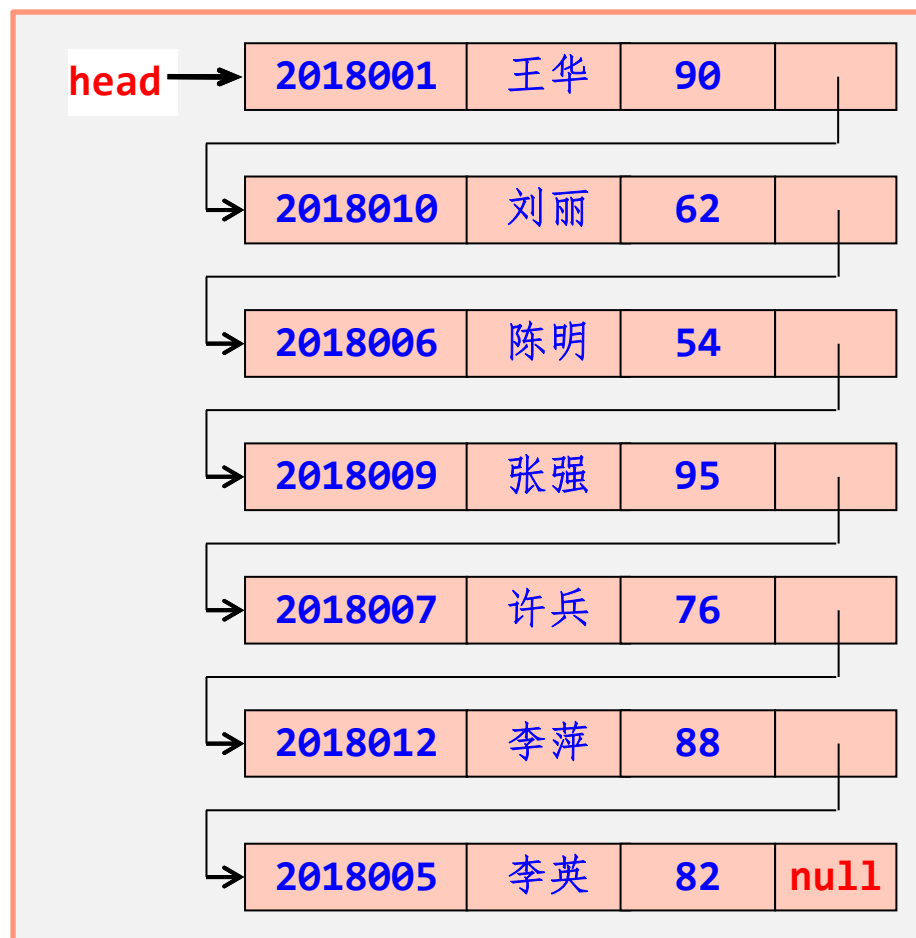
建立一个用于存放高等数学成绩表的单链表（开始结点为**head**）如下：

```
void Create()                                //创建高数成绩单链表
{
    Stud2* p2,*p3,*p4,*p5,*p6,*p7;
    head=new Stud2(2018001,"王华",90); //单链表首结点
    p2=new Stud2(2018010,"刘丽",62);  //建立其他结点
    p3=new Stud2(2018006,"陈明",54);
    p4=new Stud2(2018009,"张强",95);
    p5=new Stud2(2018007,"许兵",76);
    p6=new Stud2(2018012,"李萍",88);
    p7=new Stud2(2018005,"李英",82);
    head->next=p2;                       //建立结点之间的关系
    p2->next=p3;
    p3->next=p4;
    p4->next=p5;
    p5->next=p6;
    p6->next=p7;
    p7->next=NULL;                       //尾结点的next置为空
}
```

建立每个元素的结点

建立结点之间关系以表示对应元素的逻辑关系

用head唯一标识单链表



这种存储结构的特性：

- 数据元素存放在任意的存储单元中，这组存储单元可以是连续的，也可以是不连续的。
- 通过指针域来反映数据元素的逻辑关系。

称为链式存储结构。

在软件开发中，人们设计了各种存储结构。归纳为4种基本的存储结构。

- 顺序存储结构
- 链式存储结构
- 索引存储结构
- 哈希（散列）存储结构

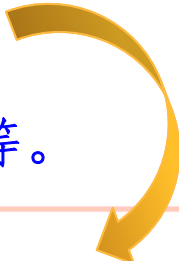
1.1.4 数据的运算



- 将数据存放在计算机中的目的是为了实现在一种或多种运算。
- 运算包括功能描述（或运算功能）和功能实现（或运算实现）。
- 前者是基于逻辑结构的，是用户定义的，是抽象的。
- 后者是基于存储结构的，是程序员用计算机语言或伪码表示的，是详细的过程，其核心是设计实现某一运算功能的处理步骤，即算法设计。

例如，对于高等数学成绩表这种数据结构，可以进行一系列的运算：

- 增加一个学生成绩记录
- 删除一个学生成绩记录
- 求所有学生的平均分
- 查找序号为*i*的学生分数等。



学号	姓名	分数
2018001	王华	90
2018010	刘丽	62
2018006	陈明	54
2018009	张强	95
2018007	许兵	76
2018012	李萍	88
2018005	李英	82

同一运算，在不同存储结构中的实现过程是不同的。



例如，查找序号为*i*的学生分数，其本身就是运算的功能描述。但在顺序存储结构和链式存储结构中的实现过程不同的。

在顺序存储结构即**data**数组中实现查找

```
int Findi(int i)                                //查找序号为i的学生分数
{  if (i<0 || i>=length)                        //i错误时返回-1
    return -1;
    return data[i].score;                       //i正确时返回分数
}
```

在链式存储结构即**head**单链表中实现查找：

```
int Findi(int i)                                //查找序号为i的学生分数
{  if (i<0) return -1;
    int j=0;
    Stud2* p=head;                             //p指向第一个结点
    while (j<i && p!=NULL)
    {  j++;
        p=p->next;
    }

    if (p==NULL)                                //i错误时返回-1
        return -1;
    else                                         //i正确时返回其分数
        return p->score;
}
```

提示

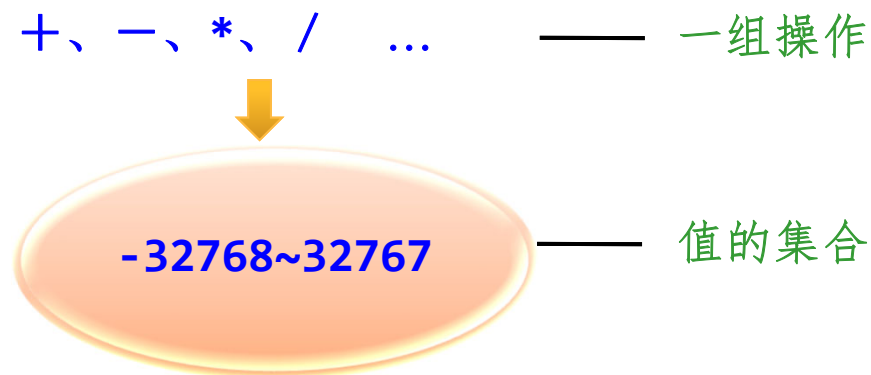
- 同一逻辑结构可以对应多种存储结构。
- 同样的运算，在不同的存储结构中，其实现过程是不同的。

1.1.5 数据结构和数据类型

1. 数据类型

数据类型是一组性质相同的值的集合和定义在此集合上的一组操作的总称。

例如，C++中的`short int` 就是整型数据类型。



- **数据结构**是指计算机处理的数据元素的组织形式和相互关系，而**数据类型**是某种程序设计语言中已实现的数据结构。
- 在程序设计语言提供的数据类型支持下，就可以根据从问题中抽象出来的各种数据模型，逐步构造出描述这些数据模型的各种新的数据结构。

2. 抽象数据类型

抽象数据类型（ADT）指的是从求解问题的数学模型中抽象出来的数据逻辑结构和运算（抽象运算），而不考虑计算机的具体实现。



ADT基本格式

ADT 抽象数据类型名

{ 数据对象：数据对象的声明

数据关系：数据关系的声明

基本运算：基本运算的声明

} **ADT** 抽象数据类型名

基本运算的声明格式：

基本运算名(参数表)：运算功能描述

【例1.6】构造集合ADT **Set**，假设其中元素为整型，遵循标准数学定义，基本运算包括：

求集合长度、求第*i*个元素、判断一个元素是否属于集合、向集合中添加一个元素、从集合中删除一个元素、复制集合和输出集合中所有元素。

另外增加3个集合运算：

求两个集合并**Union**、集合交**Inter**和集合差**Diff**。

ADT Set

{ 数据对象:

data={ d_i | $0 \leq i \leq \text{size}-1$ }

数据关系:

无

基本运算:

getsize()

get(int i)

IsIn(E e)

add(E e)

delete(E e)

Copy(s)

display()

Union(Set s2)

Inter(Set s2)

Diff(Set s2)

} ADT Set

//集合的抽象数据类型

//存放集合中元素

//返回集合的长度

//返回集合的第*i*个元素//判断*e*是否在集合中//将元素*e*添加到集合中//从集合中删除元素*e*

//返回当前集合的复制集合

//输出集合中的元素

//求 $s3=s1 \cup s2$ (*s1*为当前集合)//求 $s3=s1 \cap s2$ (*s1*为当前集合)//求 $s3=s1 - s2$ (*s1*为当前集合)

Complex

ADT



编程实现该数据结构

抽象数据类型实质上就是对一个求解问题的形式化描述（与计算机无关），程序员可以在理解基础上实现它。

1.2 算法及其描述

1.2.1 什么是算法

算法是对特定问题求解步骤的一种描述，它是指令的有限序列。

算法具有以下五个重要的特性

- **有穷性**。指算法在执行有限的步骤之后，自动结束而不会出现无限循环，并且每一个步骤在可接受的时间内完成。
- **确定性**。对于每种情况下执行的操作，在算法中都有确定的含义，不会出现二义性。并且在任何条件下，算法都只有一条执行路径。

■ **可行性**。算法的每条指令都是可执行的，即便人借助纸和笔都可以完成。

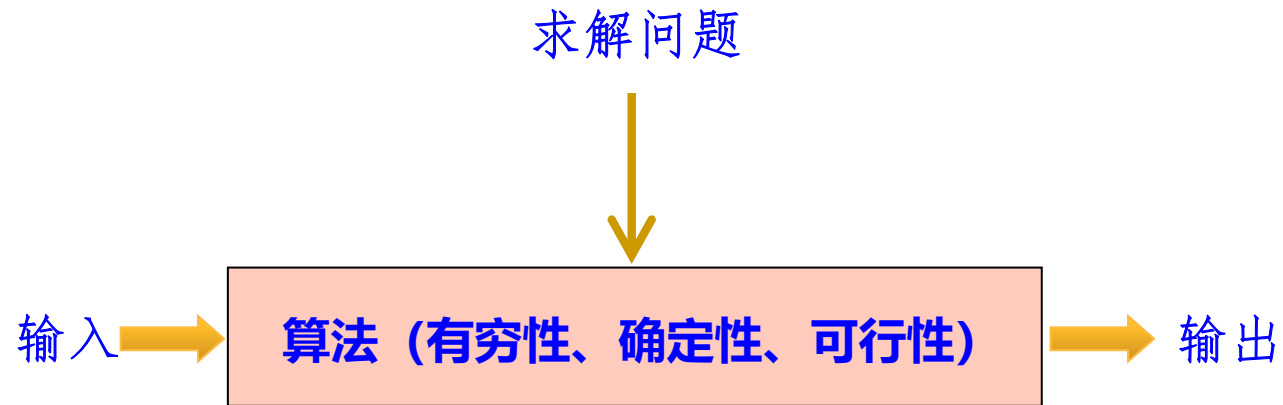
求最大公约数算法！



设两数为 a 、 b ($a \geq b$)，求 a 和 b 最大公约数 (a, b) 的步骤如下：

- (1) 用 a 除以 b ($a \geq b$)，得 $a \div b = q \dots r_1$ ($r_1 \geq 0$)。
 - (2) 若 $r_1 = 0$ ，则 $(a, b) = b$ ，结束。
 - (3) 若 $r_1 \neq 0$ ，则再用 b 除以 r_1 ，得 $b \div r_1 = q \dots r_2$ ($r_2 \geq 0$)。
 - (4) 若 $r_2 = 0$ ，则 $(a, b) = r_1$ ，结束；若 $r_2 \neq 0$ ，则继续， $\dots$ ，如此下去，直到能整除为止。
- 其最后一个余数为 0 的除数即为 (a, b) 的最大公约数。

- **输入性。**算法有零个或多个输入。大多数算法中输入参数是必要的，但对于较简单的算法，如计算 $1+2$ 的值，不需要任何输入参数，因此算法的输入可以是零个。
- **输出性。**算法至少有一个或多个输出。算法用于某种数据处理，如果没有输出，这样的算法是没有意义的，算法的输出是和输入有着某些特定关系的量。



【例1.7】 考虑下列两段描述：

(1) 描述一

```
void exam1():  
{  int n=2;  
    while(n%2==0)  
        n=n+2;  
    cout << n << endl;  
}
```

这两段描述均不能满足算法的特征，试问它们违反了哪些特性？

有穷性

(2) 描述二

```
void exam2():  
{  int x=0,y=0;  
    x=5/y  
    cout << x << endl;  
}
```

可行性

1.2.2 算法描述

通常算法用一个或者几个函数（或者方法）描述，其一般格式：

```
bool 算法对应的函数或者方法名(形参列表):
```

```
{    //临时局部变量的定义
    //实现由输入参数到输出参数的操作
    ...
}
```

} 函数体

- 函数的返回值通常为布尔类型，表示算法是否成功执行。
- 形参列表表示算法的参数，由输入参数和输出参数构成。
- 函数体实现算法的功能。

1.2.2 算法描述

模拟退火算法(simulated Annealing)

```
function SIMULATED-ANNEALING(problem, schedule) returns a solution state
  inputs: problem, a problem
           schedule, a mapping from time to “temperature”
  current  $\leftarrow$  MAKE-NODE(problem.INITIAL-STATE)
  for  $t = 1$  to  $\infty$  do
     $T \leftarrow$  schedule( $t$ )
    if  $T = 0$  then return current
    next  $\leftarrow$  a randomly selected successor of current
     $\Delta E \leftarrow$  next.VALUE – current.VALUE
    if  $\Delta E > 0$  then current  $\leftarrow$  next
    else current  $\leftarrow$  next only with probability  $e^{\Delta E/T}$ 
```

【例1.8】求和问题是当 $n \geq 1$ 时求 $s=1+2+\cdots+n$ 。设计对应的算法。

- 输入参数为 n ，操作结果为 s 。
- 初始条件是 $n \geq 1$ 。
- 当初始条件不满足，返回False，否则计算出 s 并返回True。

求解算法1

```
bool Sum1(int n,int& s)           //求和算法1
{  if (n<1) return false;
    s=n*(n+1)/2;
    return true;
}
```



```
int main()
{  int n=-5,s;
    if (Sum1(n,s))
        printf("1到%d的和=%d\n",n,s);
    else
        printf("参数n错误\n");
    return 0;
}
```

在有些情况下可以直接用算法的返回值来区分输入参数的正确性。

求解算法2

```
int Sum2(int n)                //求和算法2
{   if (n<1) return -1;
    return n*(n+1)/2;
}
```



```
int main()
{   int n=5;
    int s=Sum2(n);
    if (s!=-1)
        printf("1到%d的和=%d\n",n,s);    //输出: 1到5的和=15
    else
        printf("参数n错误\n");
    reurn 0;
}
```

在用C++语言描述算法时，用throw语句抛出异常。

求解算法3

```
int Sum3(int n)                                //求和算法3
{  if (n<1)
    throw "参数n错误";                        //抛出异常信息
    return n*(n+1)/2;
}
```



```
int main()
{  int n=-5;
    printf("1到%d的和=%d\n",n,Sum3(n));
    return 0;
}
```


1.2.3 C++语言描述算法要点

1. C++的数据类型

■ C++基本
数据类型

int

float

double

char

■ C++指针类型

每个变量都有一个内存地址，c++语言可以取变量的地址。

```
int i=2, *p=&i;
```

//i是int型变量，p是指针变量（用于存放某个整型变量的地址）

■ C++指针类型

指针变量最常用的操作是为其分配一片连续的空间

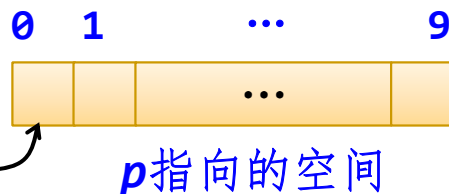
```
int* p=new int[10]; //为指针变量p分配10个int单元的空间
```



*p*本身是自动变量，
*p*指向首个int单元

p

*p*变量的空间



*p*指向的空间

*p*指向的空间必须通过
new/delete进行动态管理

2. C++函数的参数传递

■ 按值传递

```
void swap1(int x,int y)
{  int tmp=x;
   x=y;
   y=tmp;
}
```



单向值传递: $a \rightarrow x$, $b \rightarrow y$

```
int main()
{  int a=1, b=2;
   swap1(a,b);
   printf("a=%d,b=%d\n",a,b);
   return 0;
}
```

→ ? a=1,b=2

■ 引用传递

数据类型& 引用名=关联变量名;

```
double d=10;           //定义double型变量d  
double& rd=d;          //定义d的引用变量rd
```

(1) 引用参数和实参共享相同的存储空间，所以对形参的修改会影响对应实参的值，可以理解为此时实参和形参是双向值传递。

```
void swap2(int& x,int& y)
{  int tmp=x;
   x=y;
   y=tmp;
}
```



引用传递: $a \leftrightarrow x, b \leftrightarrow y$

```
int main()
{  int a=1, b=2;
   swap2(a,b);
   printf("a=%d,b=%d\n",a,b);
   return 0;
}
```

→ ? a=2, b=1

(2) 由于引用参数本身并不创建新的存储空间，所以当调用函数时传递数据量较多的情况下可以尽可能采用引用参数。

```
int fun(MyClass& s)
{   int s;
    ...
    return s;
}
```



引用参数s并不是为了实现双向传递而是节省栈空间

3. C++中的模板设计

可以包含基本数据类型，也可以包含类类型

■ 函数模板

```
template 类型形参表  
返回类型 函数名(形参表)  
{  
    函数体;  
}
```

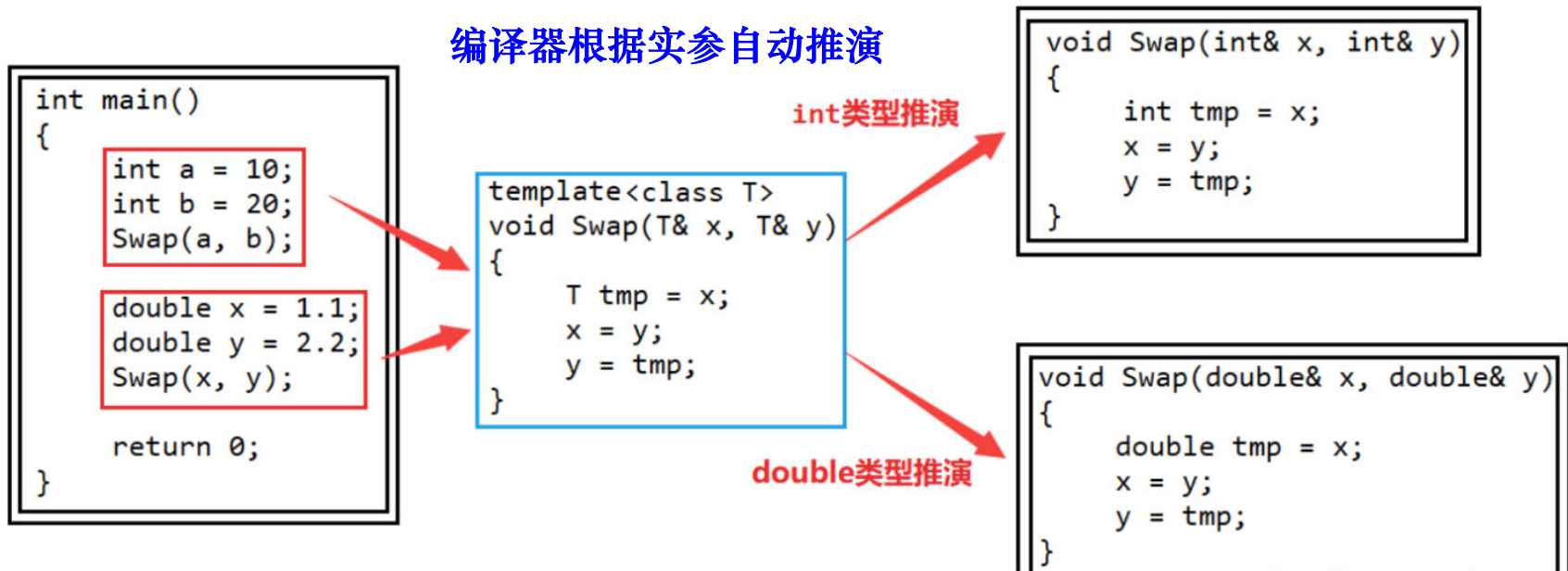
如果想处理自定义数据类型
例如类对象，就需要自己重载
操作符实现或者自己构造函数
来实现

```
template <typename T>  
T abs(T x)  
{ if (x<0) return -x;  
  return x;  
}
```

```
int main()  
{ int x=abs(-5);  
  double d=abs(-1.25);  
  return 0;  
}
```


3. C++中的模板设计

■ 函数模板



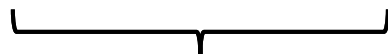
1. 这个Swap实际上不是一个函数，是一个模板，用这个模板去生成要函数
2. 我们不能调用函数模板，调用的是函数模板实例化生成的对应类型的函数
3. 编译器帮我们生成了对应的两个函数。

■ 类模板

```
template 类型形参表  
class 类模板名  
{ 类模板实现语句;  
    ...  
};
```

创建模板类的格式:

类模板名<类型实参表> 对象表;



模板类

应与该类模板中的“类型形参表”相匹配

是定义该模板类的一个或多个对象

【例1.9】 分析以下程序的功能。

```
template <typename T>
class Array
{
    T* data;
    int length;
public:
    Array(int n=1)
    {
        data=new T[n];
        length=0;
    }
    ~Array()
    {
        delete [] data;
    }
    void add(T x)
    {
        data[length]=x;
        length++;
    }
    void display()
    {
        for (int i=0;i<length;i++)
            cout << data[i] << " ";
        cout << endl;
    }
};
```

//定义类模板Array<T>
//T为类型参数，data为指针变量
//实际元素个数

//构造函数，默认参数n为1
//为data分配指向的内存空间，n为容量
//实际元素个数为0

//析构函数

//释放data指向的空间

//添加一个元素x

//输出data指向的所有元素

```
int main()
{
    Array<char> ac(3);           //定义模板类的对象ac
    ac.add('x'); ac.add('y'); ac.add('z');
    cout << "ac: ";
    ac.display();               //输出: x y z

    Array<int> ai(5);           //定义模板类的对象ai
    ai.add(1); ai.add(2); ai.add(3);
    ai.add(4); ai.add(5);
    cout << "ai: ";
    ai.display();               //输出: 1 2 3 4 5
    return 0;
}
```



说明

数据结构的实现通常用类模板来描述:

- 数据结构关注的是数据元素及其关系是如何保存的, 基于这些关系的运算是如何实现的, 而数据元素可以是任意类型。
- 使用类模板来描述, 可以避免对于具体数据元素类型的依赖。

1.3 算法分析

1.3.1 算法的设计目标

- 正确性
- 可使用性
- 可读性
- 健壮性
- 高时间性能与低存储量需求



算法分析目的：分析算法的时空效率以便改进算法性能。

1.3.2 算法时间性能分析

算法分析方式：

① **事后分析统计方法：**编写算法对应程序，统计其执行时间。

- 编写程序的语言不同
- 执行程序的环境不同
- 其他因素

所以不能用绝对执行时间进行比较。

② **事前估算分析方法：**撇开上述因素，认为算法的执行时间是问题规模 n 的函数。 ✓

问题规模 n : 一般将求解问题的输入量作为问题的规模,是问题大小的本质表示, 并用一个整数 n 来表示。问题规模 n 对不同的问题含义不同。

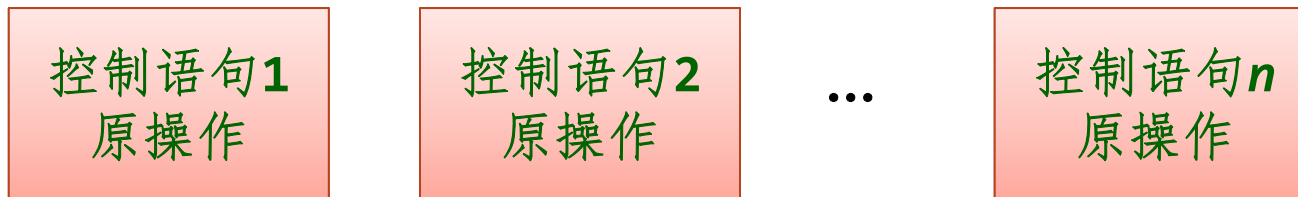
- 矩阵乘积问题的规模是矩阵的阶数。
- 一个图论问题的规模则是图中的顶点数或边数。
- 多项式运算中 n 是多项式的项数。
- 集合运算中 n 为集合中元素的个数。

显然, n 越大算法的执行时间越长。

1. 分析算法的时间复杂度

一个算法是由控制结构（顺序、分支和循环三种）和原操作（指固有数据类型的操作，如+、-、*、/、++和--等）构成的。算法执行时间取决于两者的综合效果。

一个算法的基本构成：



```
bool solve(int a[M][N],int m,int n,int& s)
{
    if (m!=n)
        return false;
    s=0;
    for (int i=0;i<n;i++)
        s+=a[i][i];
    return true;
}
```

分支结构

顺序结构

循环结构

顺序结构

原操作

- 算法的执行时间取决于控制结构和原操作的**综合效果**。
- 在一个算法中，执行原操作的次数越少，其执行时间也就相对地越少；执行原操作次数越多，其执行时间也就相对地越多。
- 算法中所有原操作的执行次数称为**算法频度**，这样一个算法的执行时间可以由算法频度来计量。

1) 计算算法频度 $T(n)$

- 假设算法的问题规模为 n ，例如对10个整数排序，问题规模为 n 就是10。算法频度是问题规模 n 的函数，用 $T(n)$ 表示。
- 算法执行时间大致等于原操作所需的时间 $\times T(n)$ ，也就是说 $T(n)$ 与算法的执行时间成正比。为此用 $T(n)$ 表示算法的执行时间。比较不同算法的 $T(n)$ 大小得出算法执行时间的好坏。

【例1.10】求两个 n 阶方阵相加 $C=A+B$ 的算法如下，求 $T(n)$ 。

```
void matrixadd(int A[N][N],int B[N][N],int  
               C[N][N],int n)  
{  for (int i=0;i<n;i++)  
    for (int j=0;j<n;j++)  
        C[i][j]=A[i][j]+B[i][j];  
}
```

执行次数

//语句① → $n+1$

//语句② → $n(n+1)$

//语句③ → n^2

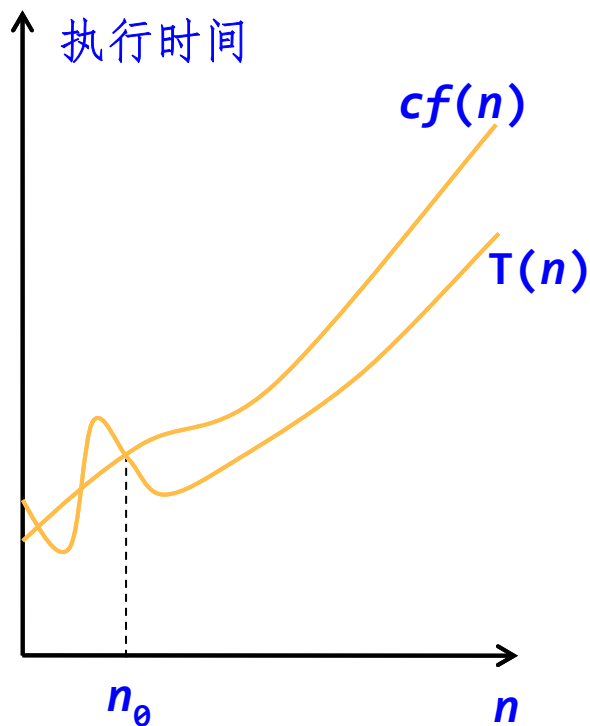


$$\begin{aligned}T(n) &= n+1+n(n+1)+n^2 \\ &= 2n^2+2n+1.\end{aligned}$$

2) $T(n)$ 采用时间复杂度表示

算法中执行时间 $T(n)$ 是问题规模 n 的某个函数 $f(n)$ ，记作：

$$T(n) = O(f(n))$$



“**O**”的形式定义为：

$T(n) = O(f(n))$ 表示存在一个正的常数 c ，使得当 $n \geq n_0$ 时都满足：

$$|T(n)| \leq c|f(n)|$$

$f(n)$ 是 $T(n)$ 的上界

这种上界可能很多，通常取最接近的上界，即**紧凑上界**

大致情况： $\lim_{n \rightarrow \infty} \frac{T(n)}{f(n)} = c$

提示

本质上讲，是一种 $T(n)$
最高数量级的比较

也就是只求出 $T(n)$ 的最高阶，忽略其低阶项和常系数，这样既可简化 $T(n)$ 的计算，又能比较客观地反映出当 n 很大时算法的时间性能。

例1.10

```
void matrixadd(int A[N][N],int B[N][N],int C[N][N],int n)
{  for (int i=0;i<n;i++)           //语句①
    for (int j=0;j<n;j++)         //语句②
        C[i][j]=A[i][j]+B[i][j]; //语句③
}
```



$$T(n)=2n^2+2n+1=O(n^2)$$



时间复杂度

一般地：

- 一个没有循环的算法的执行时间与问题规模 n 无关，记作 $O(1)$ ，也称作常数阶。
- 一个只有一重循环的算法的执行时间与问题规模 n 的增长呈线性增大关系，记作 $O(n)$ ，也称线性阶。
- 其余常用的算法时间复杂度还有平方阶 $O(n^2)$ 、立方阶 $O(n^3)$ 、对数阶 $O(\log_2 n)$ 、指数阶 $O(2^n)$ 等。

各种不同算法时间复杂度的比较关系如下：

$$O(1) < O(\log_2 n) < O(n) < O(n \log_2 n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$$

多项式阶：
P问题

指数阶：
NP问题

多项式问题 (Polynomial Problem, P问题)：可以用一个确定性算法在多项式时间内判定或解出。如果一个问题的复杂度是该问题的一个实例的规模 n 的多项式函数，则我们说这种可以在多项式时间内解决的问题属于P类问题。

非确定性多项式问题 (Nondeterministic Polynomial Problem, NP问题)：

是指一个复杂问题不能确定是否在多项式时间内找到答案，但是可以在多项式时间内验证答案是否正确。NP类问题数量很大，如图着色问题、旅行商(TSP)问题等。

NP = P? 是目前计算机科学的难题之一

```
int Sum1(int n)
{  int i,s=0;
   for (i=1;i<=n;i++)
       s+=i;
   return s;
}
```


 $O(n)$

```
int Sum2(int n)
{  int s;
   s=n*(n+1)/2;
   return s;
}
```


 $O(1)$


Sum2更好

3) 简化的算法时间复杂度分析

- 一种简化的算法时间复杂度分析方法是，仅仅考虑算法中的基本操作。
- 所谓基本操作是指算法中最深层循环内的原操作。而算法执行时间大致等于基本操作所需的时间 $\times$ 其运算次数。所以在算法分析时，计算 $T(n)$ 时仅仅考虑基本操作的执行次数。

例1.10

```
void matrixadd(int A[N][N],int B[N][N],int C[N][N],int n)
{  for (int i=0;i<n;i++)           //语句①
    for (int j=0;j<n;j++)         //语句②
        C[i][j]=A[i][j]+B[i][j]; //语句③
}
```

基本操作，执行次数为 n^2



$$T(n)=n^2=O(n^2)$$

【例1.11】 分析以下算法的时间复杂度。

```
int fun(int n)
{  int s=0
    for (int i=0;i<=n;i++)
        for (int j=0;j<=i;j++)
            for (int k=0;k<j;k++)
                s++;
    return s;
}
```

基本操作

算法频度为：

$$\begin{aligned} T(n) &= \sum_{i=0}^n \sum_{j=0}^i \sum_{k=0}^{j-1} 1 = \sum_{i=0}^n \sum_{j=0}^i (j - 1 - 0 + 1) = \sum_{i=0}^n \sum_{j=0}^i j \\ &= \sum_{i=0}^n \frac{i(i+1)}{2} = \frac{1}{2} \left(\sum_{i=0}^n i^2 + \sum_{i=0}^n i \right) = \frac{2n^3 + 6n^2 + 4n}{12} = O(n^3) \end{aligned}$$

【例1.12】 分析以下算法的时间复杂度。

```
int fun(int n)
{   int x=2;
    while (x<n/2)
        x=2*x;
    return x;
}
```



- 基本操作是语句 $x=2*x$ ，设该语句执行 m 次。
- 则 $2^{m+1} \geq n/2$ （该条件刚成立时`while`循环结束），不妨添加一个常量 k 使得 $2^{m+1} = n/2 + k$ 成立，故 $m = \log_2(n/2 + k) - 1$ 。
- $T(n) = m = \log_2(n/2 + k) - 1 = O(\log_2 n)$ 。
- 为了简单，可以直接认为 $2^{m+1} = n/2$ 成立。求出 $T(n) = m = \log_2(n/2) - 1 = \log_2 n - 2 = O(\log_2 n)$ 。

4) 时间复杂度的求和、求积定理

求和定理：假设 $T_1(n)$ 和 $T_2(n)$ 是程序段 P_1 、 P_2 的执行时间，并且有 $T_1(n)=O(f(n))$ ， $T_2(n)=O(g(n))$ 。

那么先执行 P_1 ，再执行 P_2 的总执行时间是

$T_1(n)+T_2(n)=O(\text{MAX}(f(n), g(n)))$ ，即总的时间复杂度=量级最大的程序段的时间复杂度，如多个并列循环就属于这种情况。

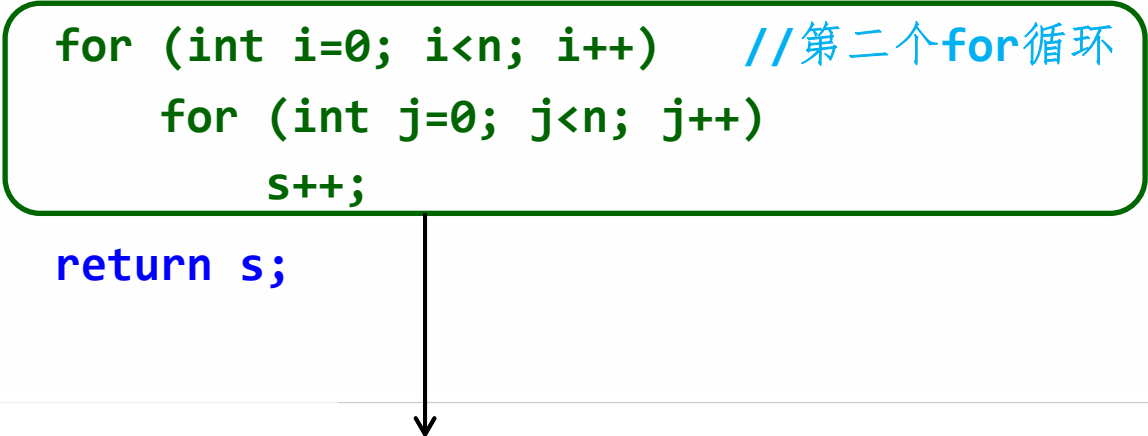

```
int fun(int n)
{  int s=0;
   for (int i=0; i<n; i++)    //第一个for循环
       s+=2;
   for (int i=0; i<n; i++)    //第二个for循环
       for (int j=0; j<n; j++)
           s++;
   return s;
}
```

$$T(n)=O(\text{MAX}(n, n^2))=O(n^2)。$$

求积定理：假设 $T_1(n)$ 和 $T_2(n)$ 是程序段 P_1 、 P_2 的执行时间，并且有 $T_1(n)=O(f(n))$ ， $T_2(n)=O(g(n))$ 。

那么， $T_1(n) \times T_2(n) = O(f(n) \times g(n))$ ，如嵌套代码的复杂度=嵌套内外代码复杂度的乘积。

```
int fun(int n)
{ int s=0;
  for (int i=0; i<n; i++) //第一个for循环
    s+=2;
  for (int i=0; i<n; i++) //第二个for循环
    for (int j=0; j<n; j++)
      s++;
  return s;
}
```



$$T2(n)=O(n) \times O(n)=O(n^2)$$

2. 算法的最好、最坏和平均时间复杂度

定义： 设一个算法的输入规模为 n ， D_n 是所有输入实例的集合，任一输入 $I \in D_n$ ， $P(I)$ 是 I 出现的概率，有 $\sum_{I \in D_n} P(I) = 1$ ， $T(I)$ 是算法在输入 I 下的执行时间，则算法的平均时间复杂度为：

$$A(n) = \sum_{I \in D_n} P(I) \times T(I)$$

例如，10个1~10的整数序列递增排序：

$n=10$

$I_1=\{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$

$I_2=\{2, 1, 3, 4, 5, 6, 7, 8, 9, 10\}$

...

$I_m=\{10, 9, 8, 7, 6, 5, 4, 3, 2, 1\}$

构成 D_n , $P(I)=1/m$

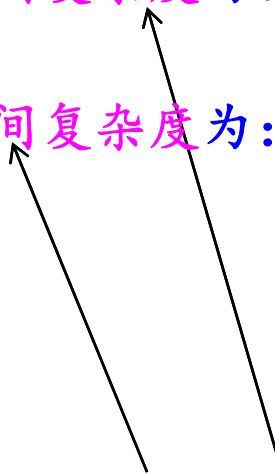


所有可能的初始序列有 m 个, $m=10!$

算法的最坏时间复杂度为： $W(n)=\text{MAX}\{T(I)\}$
 $I \in D_n$

算法的最好时间复杂度为： $B(n)=\text{MIN}\{T(I)\}$
 $I \in D_n$

一种或几种特殊情况





提示

算法时间性能比较：假如求同一问题有两个算法：**A**和**B**，如果算法**A**的平均时间复杂度为 $O(n)$ ，而算法**B**的平均时间复杂度为 $O(n^2)$ 。

一般情况下，认为算法**A**的时间性能好比算法**B**。

【例1.13】 以下算法用于在数组 $a[0..n-1]$ 查找元素 k ，假设 k 总是包含在 a 中，分析算法的最好、最坏和平均时间复杂度。

```
int findk(int a[],int n,int k)
{  int i=0;
   while (i<n && a[i]!=k)
       i++;
   return i;
}
```



```
int findk(int a[],int n,int k)
{
    int i=0;
    while (i<n && a[i]!=k)
        i++;
    return i;
}
```



该算法的主要时间花费在元素比较上，可以将元素比较看成基本操作。

(1) 算法在查找中总是从 $i=0$ 开始的，如果 $a[0]=k$ ，则仅仅一次比较就成功找到 k ，呈现最好情况，所以算法的最好时间复杂度为 $O(1)$ 。

```
int findk(int a[],int n,int k)
{  int i=0;
   while (i<n && a[i]!=k)
       i++;
   return i;
}
```

(2) 如果 $a[n-1]=k$ ，则需要 n 次比较成功找到 k ，呈现最坏情况，所以算法的最坏时间复杂度为 $O(n)$ 。

```
int findk(int a[],int n,int k)
{   int i=0;
    while (i<n && a[i]!=k)
        i++;
    return i;
}
```

(3) 考虑平均情况:

$a[0]=k$ 时比较1次

$a[1]=k$ 时比较2次

...

$a[n-1]=k$ 时比较 n 次

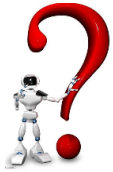
共 n 种情况, 假设等概率, 也就是说每种情况的概率为 $1/n$, 则平均比较次数 $= (1+2+\cdots+n)/n = (n+1)/2 = O(n)$, 所以算法平均时间复杂度为 $O(n)$ 。

1.3.3 算法空间性能分析

- 一个算法的存储量包括形参所占空间和临时变量所占空间。在对算法进行存储空间分析时，只考察临时变量所占空间。
- 空间复杂度是对一个算法在运行过程中临时占用的存储空间大小的量度，一般也作为问题规模 n 的函数，以数量级形式给出，记作： $S(n)=O(g(n))$ 。
- 其中“ O ”的含义与时间复杂度分析中的相同。

```
int Max(int a[],int n)
{  int maxi=0;
   for (int i=1;i<n;i++)
       if (a[i]>a[maxi])
           maxi=i;
   return a[maxi];
}
```

函数体内分配的变量空间为临时空间，不计形参占用的空间，这里的仅计*i*、**maxi**变量的空间。



为什么算法空间分析只考虑临时空间，而不必考虑形参的空间呢？

```
int Max(int a[],int n)
{  int maxi=0;
   for (int i=1;i<n;i++)
       if (a[i]>a[maxi])
           maxi=i;
   return a[maxi];
}
```



```
void Maxfun()
{  int b[]={1,2,3,4,5};
   int n=5;
   printf("Max=%d\n",Max(b,n));
}
```

如果Max函数中再考虑形参**a**的空间，就重复累计了执行整个算法所需的空间。



传递数组地址

Maxfun算法中为**b**数组分配了相应的内存空间，其空间复杂度为 $O(n)$

【例1.14】 分析例1.10～例1.13算法的空间复杂度。

例1.10

```
void matrixadd(int A[M][N],int B[M][N],int C[M][N],int n)
{  for (int i=0;i<n;i++)                //语句①
    for (int j=0;j<m;j++)                //语句②
        C[i][j]=A[i][j]+B[i][j];        //语句③
}
```



空间复杂度为 $O(1)$

例1.11

```
int fun(int n)
{  int s=0
    for (int i=0;i<=n;i++)
        for (int j=0;j<=i;j++)
            for (int k=0;k<j;k++)
                s++;
    return s;
}
```



空间复杂度为 $O(1)$

例1.12

```
int fun(int n)
{   int x=2;
    while (x<n/2)
        x=2*x;
    return x;
}
```



空间复杂度为 $O(1)$

例1.13

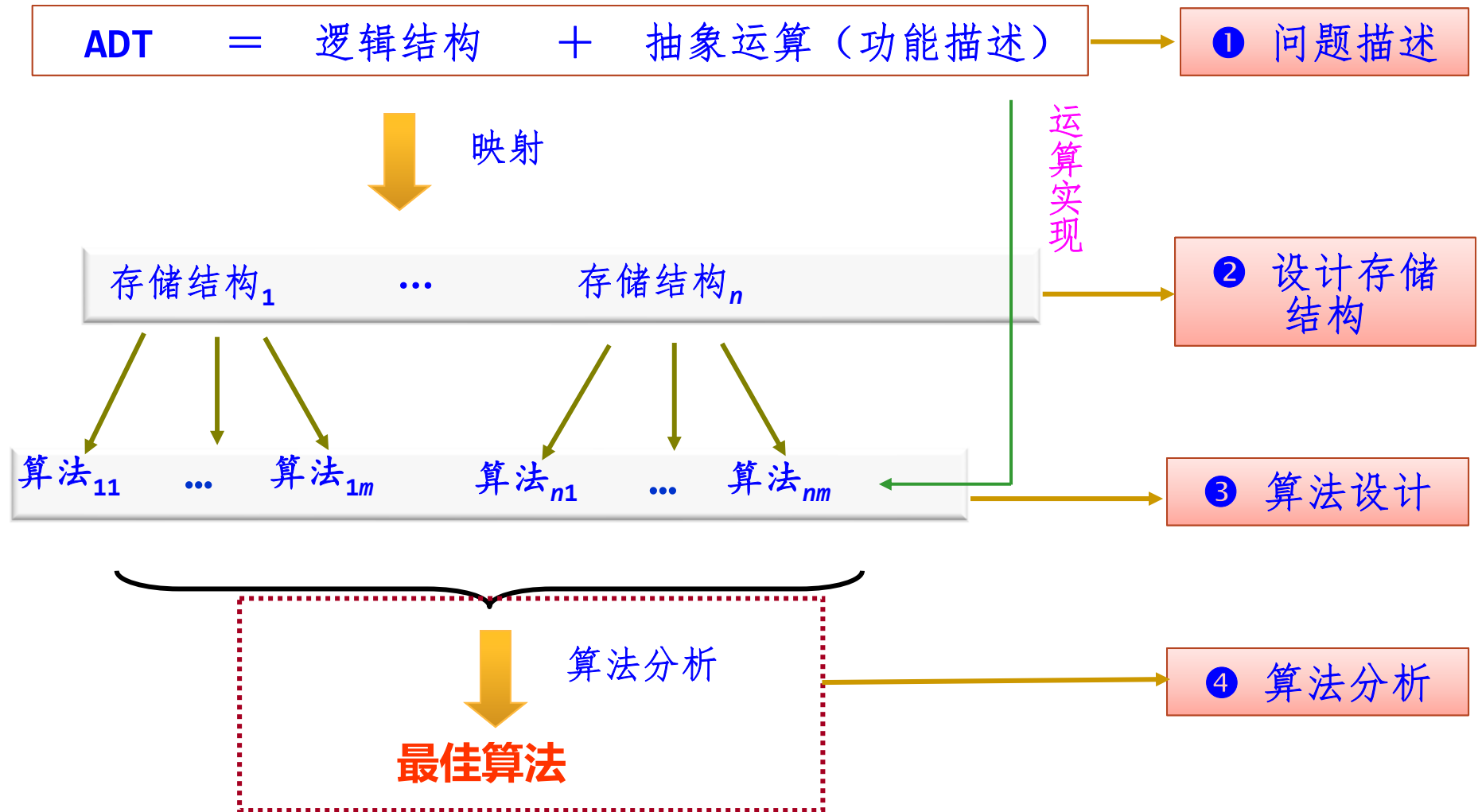
```
int findk(int a[],int n,int k)
{  int i=0;
   while (i<n && a[i]!=k)
       i++;
   return i;
}
```



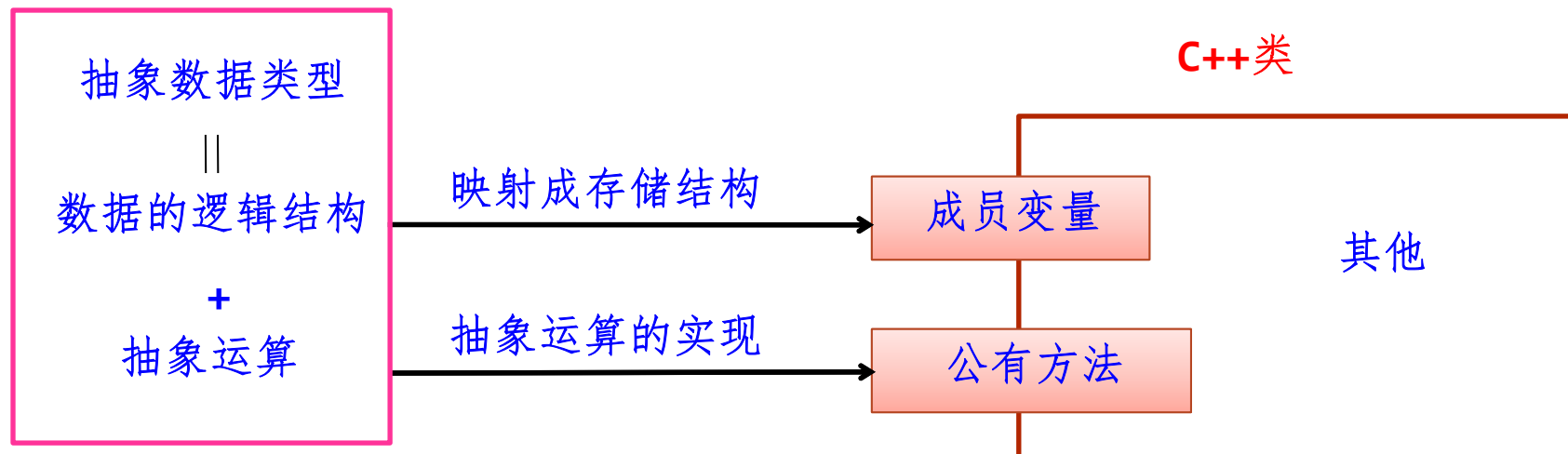
空间复杂度为 $O(1)$

1.4 数据结构的目标

好算法设计的过程



采用C++语言实现抽象数据类型时，通常将一个抽象数据类型设计成一个C++类，采用类的数据变量表示数据的存储结构，将抽象运算通过类的公有方法实现。



存储结构对算法的影响主要在两方面：

- 存储结构的存储能力
- 存储结构应与所选择的算法相适应

【例1.15】 设计一个完整的程序实现例1.6的抽象数据类型，并用相关数据进行测试。

问题描述

【例1.6】 构造集合ADT Set，假设其中元素为整型，遵循标准数学定义，基本运算包括：

求集合长度、求第*i*个元素、判断一个元素是否属于集合、向集合中添加一个元素、从集合中删除一个元素、复制集合和输出集合中所有元素。

另外增加3个集合运算：

求两个集合并Union、集合交Inter和集合差Diff。

ADT Set

{ 数据对象:

data={ d_i | $0 \leq i \leq \text{size}-1$ }

数据关系:

无

基本运算:

getsize()

get(int i)

IsIn(E e)

add(E e)

delete(E e)

Copy(s)

display()

Union(Set s2)

Inter(Set s2)

Diff(Set s2)

} ADT Set

//集合的抽象数据类型

//存放集合中元素

//返回集合的长度

//返回集合的第*i*个元素

//判断*e*是否在集合中

//将元素*e*添加到集合中

//从集合中删除元素*e*

//返回当前集合的复制集合

//输出集合中的元素

//求 $s3=s1 \cup s2$ (*s1*为当前集合)

//求 $s3=s1 \cap s2$ (*s1*为当前集合)

//求 $s3=s1 - s2$ (*s1*为当前集合)

📁 设计存储结构

```
class Set                //集合类
{  int* data;            //动态data存放集合元素
  int length;           //length为集合的长度
  ...
};
```


设计运算算法

Set类包含以下基本运算方法：

```
public:
    Set()                                //构造函数
    {   data=new int[MaxSize];          //data存放集合元素
        length=0;                       //length为集合的长度
    }

    ~Set()                               //析构函数
    {
        delete[] data;
    }
```

```
int getlength()  
{  
    return length;  
}
```

//返回集合的长度

```
int get(int i)  
{ if (i<0 || i>=length)  
    throw "参数i错误";  
    return data[i];  
}
```

//返回集合的第*i*个元素

//检测参数*i*的正确性

```
bool IsIn(int e)  
{ for (int i=0;i<length;i++)  
    if (data[i]==e) return true;  
    return false;  
}
```

//判断*e*是否在集合中

```
void add(int e)
{  if (!IsIn(e))
    {  data[length]=e;
        length++;
    }
}
```

//将元素e添加到集合中
//元素e不在集合中

```
void delelem(int e)
{  int i=0;
    while (i<length && data[i]!=e) i++;
    if (i>=length)
        return;
    for (int j=i+1;j<length;j++)
        data[j-1]=data[j];
    length--;
}
```

//从集合中删除元素e

//未找到元素e直接返回
//找到元素e后通过移动实现删除

```
Set& Copy()                //返回当前集合的复制集合
{
    static Set s1;
    for (int i=0;i<length;i++)
        s1.data[i]=data[i];
    s1.length=length;
    return s1;
}

void display()              //输出集合中的元素
{
    for (int i=0;i<length-1;i++)
        printf("%d ",data[i]);
    printf("%d\n",data[length-1]);
}
```

$s3 = s1 \cup s2$:

- $s3 = s1$
- 将 $s2$ 中不属于 $s1$ 的元素添加到 $s3$



```
Set& Union(Set& s2)           //求 $s3 = s1 \cup s2$  ( $s1$ 为当前集合)
{   Set& s3 = Copy();         //将当前集合复制到 $s3$ 
    for (int i=0; i<s2.getLength(); i++)
        //将 $s2$ 中不在 $s1$ 中的元素添加到 $s3$ 中
    {   int e = s2.get(i);
        if (!IsIn(e)) s3.add(e);
    }
    return s3;                //返回 $s3$ 
}
```

$s3 = s1 \cap s2$:

- $s3 = \{\}$
- 将 $s1$ 中属于 $s2$ 的元素添加到 $s3$



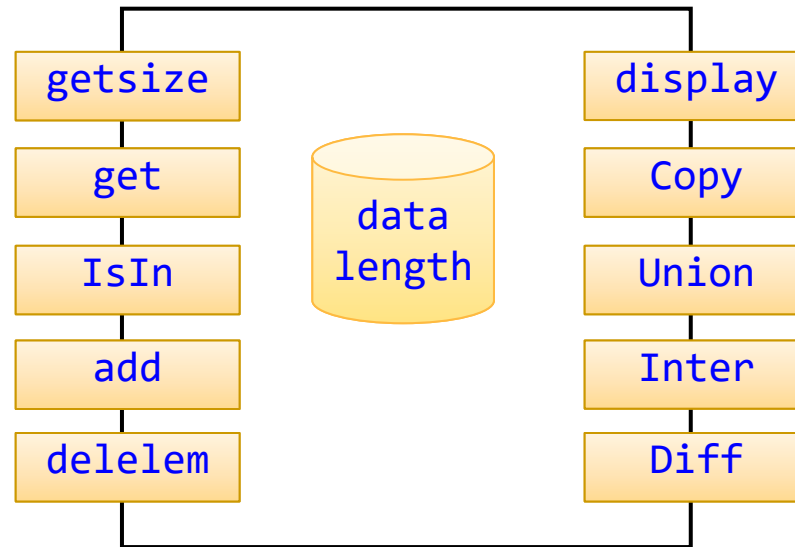
```
Set& Inter(Set& s2)           //求 $s3 = s1 \cap s2$  ( $s1$ 为当前集合)
{
    static Set s3;
    for (int i=0; i<length; i++)
        //将 $s1$ 中出现在 $s2$ 中的元素复制到 $s3$ 中
        {
            int e=data[i];
            if (s2.IsIn(e)) s3.add(e);
        }
    return s3;                 //返回 $s3$ 
}
```

s3=s1-s2:

- s3={}
 - 将s1中不属于s2的元素添加到s3
- 

```
Set& Diff(Set& s2)                //求s3=s1-s2(s1为当前集合)
{ static Set s3;
  for (int i=0;i<length;i++)//将s1中不出现在s2中的元素复制到s3中
  { int e=data[i];
    if (!s2.IsIn(e)) s3.add(e);
  }
  return s3;                      //返回s3
}
```

Set类的描述



设计主程序

```
int main()
{
    Set s1,s2;
    s1.add(1);
    s1.add(4);
    s1.add(2);
    s1.add(6);
    s1.add(8);
    printf("集合s1: "); s1.display();
    printf("s1的长度为%d\n",s1.getlength());
    s2.add(2);
    s2.add(5);
    s2.add(3);
    s2.add(6);
    printf("集合s2: "); s2.display();
}
```

```
printf("集合s1和s2的并集->s3\n");  
Set& s3=s1.Union(s2);  
printf("集合s3: "); s3.display();  
printf("集合s1和s2的差集->s4\n");  
Set& s4=s1.Diff(s2);  
printf("集合s4: "); s4.display();  
printf("集合s1和s2的交集->s5\n");  
Set& s5=s1.Inter(s2);  
printf("集合s5: "); s5.display();  
return 0;  
}
```

程序执行结果

```
集合s1: 1 4 2 6 8  
s1的长度为5  
集合s2: 2 5 3 6  
集合s1和s2的并集->s3  
集合s3: 1 4 2 6 8 5 3  
集合s1和s2的差集->s4  
集合s4: 1 4 8  
集合s1和s2的交集->s5  
集合s5: 2 6
```